Iniziato	martedì, 17 giugno 2025, 09:58
Stato	Completato
Terminato	martedì, 17 giugno 2025, 10:58
Tempo impiegato	1 ora

Domanda 1

Completo

Punteggio max.: 6,00

Describe, formally or informally, a deterministic TM of the kind you prefer, which decides the following language:

$$\square = \{w \mid w \text{ starts with } 00 \text{ and ends with } 1\}$$

Examples of strings which are in $\square$ are 001 and 001011, while $01 \notin \square$. Study the complexity of TM you have defined.

```
STATES = {q_init, q_reject, q_accept}

ALPHABET = {0,1, > (start), _ (blank)}

TRANSITION FUNCTION:

(q_init, >) -> (q_read, >, R)

(q_read, 0) -> (q_0, >, R)
(q_read, 1) -> (q_reject, 1, S)
(q_read, _) -> (q_reject, _, S)

(q_0, 0) -> (q_00, 0, R)
(q_0, 1) -> (q_reject, 1, S)
(q_0, _) -> (q_reject, _, S)

(q_00, 0) -> (q_00, 0, R)
(q_00, 1) -> (q_1, 1, R)
(q_00, _) -> (q_reject, _, S)

(q_1, 0) -> (q_00, 0, R)
(q_1, 1) -> (q_1, 1, R)
(q_1, _) -> (q_accept, _, S)

The complexity of my TM is linear as it scans the input tape once.
```

Domanda 2

Completo

Punteggio max.: 7,00

You are required to prove that the following problem ☐ is in $\mathbf{NP}$. To do that, you can give a TM or define some pseudocode. The language ☐ includes precisely those binary strings which are encodings of natural numbers which can be factored into 4 distinct prime numbers. In doing so you can assume that primality can be tested in polynomial time. An example of a number which is in ☐ is $210 = 2 \cdot 3 \cdot 5 \cdot 7$.

In order to prove L to be in NP we provide a certificate and a verifier that verifies the correctness of the certificate in polynomial time.

```
CERTIFICATE: list of natural numbers N
VERIFIER:
for n in N:
    factors = {}
    actual_num = n
    for num in {2..n}:
        if is_prime(num) and (actual_num%num==0):
            actual num /= num
            add num to factors
            if len(factors) > 4:
                return False      # we assume exactly 4 prime numbers
    if actual_num != 1:
        return False
return True
```

It works in O(n) for each number in input and in O(n*|N|) if we consider a list of N numbers.

Commento:
The proposed solution does not make too much sense. What's the role of the certificate. Where's the input.

Domanda 3

Completo

Punteggio max.: 7,00

Please recall that given an undirected graph $\mathbb{G}$, a cover of it is a subset of its nodes such that every edge of $\mathbb{G}$ contains one of the nodes in the subset. Consider now a variation to the vertex cover problem, called $\mathrm{FOURCOVER}$, which contains all undirected graphs having a cover of cardinality equal to 4. To which complexity class does this new problem belong? Prove your claim.

```
Prove it is in NP.
INPUT: G=(V,E)
CERTFICATE: cover C subset of V
VERIFIER:
if |C| != 4:
  return False
for v in C:
    if v not in V:              # check C subset of V
      return False
    count = 0
    for (a,b) in E:
        if (a==v and b!=v and (b in C)) or (b==v and a!=v and (a in C)):
          count += 1
    if count != 4:
      return False
return True
It works in O(|C|*|E|) = O(|V|*|E|)
```

Prove it is NP-hard.

We can reduce it from the cover by finding f in polytime so that f(G) is FOURCOVER <==> G is COVER.

We can prove it by adding to G 4 nodes and connecting them to each node of V, in this way the COVER solution is untouched and we are sure that those 4 nodes are a solution for FOURCOVER.

The other way is trivial as a FURCOVER is a COVER.

Commento:

The problem is actually in P. Moreover, the proposed pseudocode does not seem to do what it is supposed to do...

◀ ICC - 17062025 - QUESTIONS

Vai a...